Laboratorio de Sistemas Operativos
Gestión de Memoria

Práctica No. 3

Fecha de inicio:	Abr. 20 de 2026
Fecha de entrega:	May. 03 de 2026 (23:59)
Medio de entrega:	Moodle

Objetivo

Esta práctica evalúa los conceptos fundamentales de la visualización de memoria en sistemas operativos modernos. A través de programas en C y herramientas de Linux, el estudiante comprobará empíricamente los mecanismos que el SO utiliza para ofrecer a cada proceso la ilusión de un espacio de direcciones privado y contiguo.

*** Herramientas útiles ***

Compilador: `gcc -Wall -o nombre nombre.c`
Valgrind: `valgrind --leak-check=full ./nombre`

Sino cuenta con Valgrind, instalarlo: `sudo apt install valgrind`

1 Espacio de direcciones

Tema libro de texto: `vm-intro` — The Abstraction: Address Spaces

El SO crea para cada proceso la abstraccion de un **espacio de direcciones virtual** privado, compuesto por tres regiones principales: código (*text*), *heap* y *stack*. Linux expone esta información en el pseudo-sistema de archivos **`/proc`**.

1.1 Actividad: Programa base

Guarde el siguiente código como `mem_map.c` y compilelo:

```c
// mem_map.c
#include <stdio.h>
```

```c
#include <stdlib.h>
#include <unistd.h>

int global_var = 42;        /* segmento de datos (.data) */

int main() {
    int  local_var = 10;                        /* stack */
    int *heap_var  = malloc(sizeof(int)*100);  /* heap  */
    *heap_var = 99;

    printf("PID del proceso     : %d\n",  getpid());
    printf("Dir. codigo (main)  : %p\n",  (void*)main);
    printf("Dir. global_var     : %p\n",  (void*)&global_var);
    printf("Dir. local_var      : %p\n",  (void*)&local_var);
    printf("Dir. heap_var       : %p\n",  (void*)heap_var);

    printf("\nPresione ENTER para continuar...\n");
    getchar();
    free(heap_var);
    return 0;
}
```

Code 1: mem_map.c

1.2 Actividad: Visualizar los mapas de memoria de un proceso

Mientras el programa espera el ENTER, abra una segunda terminal:

```
# Leer el mapa de memoria del proceso
cat /proc/$(pgrep mem_map)/maps

# Vista resumen con tamanos de cada region
pmap -x $(pgrep mem_map)
```

1.3 Actividad: Exploración de `/proc/[pid]/maps`

1. Identifique en la salida de `/proc/maps` las regiones *text*, *heap* y *stack*. ¿Qué permisos (`r/w/x/p`) tiene cada una? ¿Por qué difieren?

2. Compare las direcciones impresas con los rangos de `/proc/maps`. ¿A qué región pertenece cada variable?

3. ¿Qué otras regiones aparecen en el mapa (`libc`, `[vdso]`, `[vsyscall]`)? ¿Que funcion cumple cada una?

4. ¿Son las direcciones virtuales iguales a las fisicas? Explique apoyandose en el concepto de *address space* del OSTEP.

1.4 Actividad: Comparar espacios de dos procesos simultáneos

Ejecute dos instancias de `mem_map` al mismo tiempo en dos terminales distintas y compare las direcciones de `main`, `global_var` y `heap_var`.

1. ¿Son las mismas direcciones virtuales en ambos procesos? ¿Qué conclusión saca sobre el aislamiento del espacio de direcciones?

2. ¿Podría el Proceso A leer o modificar la variable global del Proceso B mediante su dirección virtual? Justifique.

2 API de Memoria

Tema libro de texto: `vm-api` — Interlude: Memory API

En C la memoria del heap se administra explicitamente con `malloc()`, `calloc()`, `realloc()` y `free()`. Los errores mas comunes son: *memory leak* (no liberar), *double free* (liberar dos veces), *buffer overflow* (escribir fuera del bloque asignado) y *use-after-free* (acceder a memoria ya liberada). **Valgrind** detecta todos estos errores en tiempo de ejecución.

2.1 Actividad: Programa base

Cree el archivo `heap_demo.c` y compilelo:

```c
// heap_demo.c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n    = 10;
    int *arr = (int *) malloc(n * sizeof(int));
    if (arr == NULL) { perror("malloc"); return 1; }

    for (int i = 0; i < n; i++) arr[i] = i * i;
    printf("Arreglo original:  ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");

    /* Redimensionar a 20 enteros */
    arr = (int *) realloc(arr, 20 * sizeof(int));
    if (arr == NULL) { perror("realloc"); return 1; }
    for (int i = n; i < 20; i++) arr[i] = i * i;
```

```c

        printf("Arreglo ampliado: ");
        for (int i = 0; i < 20; i++) printf("%d ", arr[i]);
        printf("\n");

        free(arr);
        return 0;
    }
```

Code 2: heap_demo.c

Una vez compilado, monitoree la ejecución del programa con Valgrind:

```
gcc -Wall -o heap_demo heap_demo.c
valgrind --leak-check=full --track-origins=yes ./heap_demo
```

2.2 Actividad: Uso correcto de `malloc` y `free`

1. Muestre la salida completa de Valgrind. ¿Reporta errores o fugas de memoria? ¿Que significa el mensaje `"All heap blocks were freed"`?

2. ¿Por qué se usa `sizeof(int)` en lugar del valor literal `4`? ¿Qué ventaja ofrece en portabilidad entre arquitecturas?

3. ¿Qué devuelve `malloc` Cuándo no hay memoria disponible? ¿Por qué es critico verificar ese valor antes de usarlo?

2.3 Actividad: Código con *bugs* de memoria

El siguiente código tiene **tres errores clásicos**. Guarde como `buggy_mem.c`:

```c
    // buggy_mem.c  --  NO ejecutar sin Valgrind
    #include <stdio.h>
    #include <stdlib.h>
    #include <string.h>

    int main() {
        /* ERROR 1: buffer overflow */
        int *p = malloc(5 * sizeof(int));
        for (int i = 0; i <= 5; i++)   /* <= en vez de < */
        p[i] = i;

        /* ERROR 2: memory leak (nunca se llama free(q)) */
        char *q = malloc(100);
        strcpy(q, "hola mundo");
        printf("%s\n", q);

```

```
        /* ERROR 3: use-after-free */
        free(p);
        printf("p[0] = %d\n", p[0]);   /* acceso ilegal */

        return 0;
    }
```

Code 3: buggy_mem.c — contiene errores intencionales

Compile y ejecute el programa:

```
gcc -Wall -g -o buggy_mem buggy_mem.c
valgrind --leak-check=full --track-origins=yes ./buggy_mem
```

2.4 Actividad: Identificar y corregir errores de memoria

1. Transcriba los mensajes que arroja Valgrind. ¿Cuál mensaje corresponde a cada uno de los tres errores clásicos?
2. Corrija el programa (`buggy_mem_fixed.c`) y verifique con Valgrind que no queda ningún error ni fuga.
3. ¿Qué consecuencias puede tener un *use-after-free* en un programa real en términos de seguridad y estabilidad del sistema?

3 Traducción de direcciones — Base & Bounds

Tema del libro: `vm-mechanism` — Mechanism: Address Translation

El mecanismo mas simple de traducción de direcciones utiliza dos registros de hardware: **base** (dirección física donde inicia el proceso) y **bounds** (tamaño del espacio asignado). La formula de traducción es:

$$\text{PA} = \text{VA} + \text{base}, \qquad \text{valida solo si} \quad 0 \leq \text{VA} < \text{bounds}$$

Si la condición no se cumple, el hardware lanza una excepción que el SO atiende como una violación de segmento (*segmentation fault*).

3.1 Actividad: Simulador

Guarde como `base_bounds.c`, aquí **`VA`** hace re referencia a la dirección virtual y **`PA`** a la dirección física:

```
    // base_bounds.c
    #include <stdio.h>

    typedef struct { int base; int bounds; } Registro;
```

```c

/* Traduce VA -> PA; imprime excepcion si viola bounds */
int traducir(Registro r, int va) {
    if (va < 0 || va >= r.bounds) {
        printf("  [EXCEPCION] VA=%d viola bounds=%d\n",
        va, r.bounds);
        return -1;
    }
    return r.base + va;
}

int main() {
    Registro procA = {32,  64};   /* base=32,  bounds=64 */
    Registro procB = {128, 80};   /* base=128, bounds=80 */
    int vas[] = {0, 10, 63, 64, 100};
    int n = sizeof(vas) / sizeof(vas[0]);

    printf("--- Proceso A (base=%d, bounds=%d) ---\n",
        procA.base, procA.bounds);
    for (int i = 0; i < n; i++) {
        int pa = traducir(procA, vas[i]);
        if (pa != -1)
        printf("  VA=%3d  ->  PA=%3d\n", vas[i], pa);
    }
    printf("--- Proceso B (base=%d, bounds=%d) ---\n",
        procB.base, procB.bounds);
    for (int i = 0; i < n; i++) {
        int pa = traducir(procB, vas[i]);
        if (pa != -1)
        printf("  VA=%3d  ->  PA=%3d\n", vas[i], pa);
    }
    return 0;
}
```

Code 4: base_bounds.c

3.2 Actividad: Base & Bounds — Análisis

1. Compile y ejecute. Muestre la salida completa. ¿Que ocurre al acceder a `VA=64` y `VA=100` en el Proceso A? ¿Que haria el SO real ante esta excepcion?

2. Agregue un Proceso C (`base=0, bounds=32`) al programa y traduzca las mismas VAs. ¿Puede el Proceso A acceder a las direcciones del Proceso C directamente? Justifique.

3. ¿Cuál es la limitacion principal del esquema base & bounds que motiva el surgimiento de la segmentacion?

4 Segmentacion

Tema del libro: `vm-segmentation` — Segmentation

La segmentación divide el espacio de direcciones en segmentos lógicos independientes (*code*, *heap*, *stack*), cada uno con su propio par base/bounds. Los **bits mas significativos** de la VA identifican el segmento; los bits restantes forman el **offset**.

4.1 Traducción manual con tabla de segmentos

Dado un espacio de **14 bits** (2 bits de selector + 12 bits de offset), use la siguiente tabla de segmentos:

Segmento	Base (PA)	Tamaño	Crece	Selector
Code	0x4000 (16 384 B)	2 KB	positivo →	`00`
Heap	0x6000 (24 576 B)	3 KB	positivo →	`01`
Stack	0x2800 (10 240 B)	2 KB	negativo ←	`11`

Traduzca manualmente y complete la tabla:

VA (hex)	Selector	Offset	Segmento	PA o Excepcion
`0x03A0`	`00`	`0x3A0`	Code	
`0x1800`	`01`	`0x800`	Heap	
`0x3C00`	`11`	`0xC00`	Stack	
`0x0C00`	`00`	`0xC00`	Code	
`0x2200`	`10`	—	???	

1. Muestre el cálculo paso a paso para cada VA.
2. ¿Por qué el Stack crece en dirección negativa? ¿Que ajuste especial requiere la formula al calcular el PA?
3. ¿Qué ventaja tiene la segmentación frente a base & bounds en cuanto a utilizacion de la memoria física?
4. ¿Qué es la *fragmentación externa*? ¿Por qué surge con segmentación? Ilustre con un diagrama de bloques de memoria.

5 Paginación

Tema del libro: `vm-paging` — Introduction to Paging

La paginación divide el espacio virtual en **páginas** de tamaño fijo y la memoria física en **marcos** (*frames*) del mismo tamaño. Una **tabla de páginas** mapea cada VPN al PFN correspondiente. La descomposición de la VA es:

$$\text{PA} = \underbrace{\text{PFN}}_{\text{de la tabla}} \times \text{PageSize} + \underbrace{\text{offset}}_{\text{bits bajos de VA}}$$

Acá hay tres definiciones útiles:

1. **VPN**: Virtual Page Number (Número de Página Virtual)
   Es la "dirección" de una página dentro del espacio virtual del proceso. Cuándo el proceso genera una dirección virtual, los bits más altos son el VPN y le dicen al hardware qué página está buscando. Es como el número de un apartamento en un edificio: identifica la unidad, pero no dice en qué piso físico está. Ejemplo: con páginas de 4 KB (12 bits de offset) y una VA de 32 bits, los 20 bits superiores son el VPN → hay hasta 1.048.576 páginas virtuales posibles.

2. **PFN**: Physical Frame Number (Número de Marco Físico)
   Es el equivalente físico del VPN: identifica un bloque concreto de la RAM real. La memoria física se divide en "marcos" (frames) del mismo tamaño que las páginas, y cada marco tiene su número. El trabajo de la tabla de páginas es precisamente decir: "la página virtual número X está en el marco físico número Y", es decir, VPN → PFN.

3. **PTE**: Page Table Entry (Entrada de la Tabla de Páginas)
   Es una fila de la tabla de páginas. Cada página virtual tiene su propia PTE, y dentro de ella se almacenan dos cosas principales: El PFN dice a qué marco físico corresponde esa página y contiene varios bits de control con información útil.

5.1 Actividad: Cálculo de la tabla de páginas

Considere el sistema:

Parámetro	**Valor**
Espacio virtual	32 bits
Tamano de pagina	4 KB = 2^{12} bytes
Espacio físico	20 bits (1 MB)
Tamaño de PTE	4 bytes

1. ¿Cuántos bits se necesitan para el VPN y Cuántos para el offset? Muestre el cálculo.

2. ¿Cuántas entradas tiene la tabla de páginas de un proceso?
3. ¿Cuanta memoria ocupa la tabla de paginas completa? ¿Es razonable ese tamaño para cada proceso?
4. ¿Cuántos bits necesita el PFN dentro de la PTE? ¿Qué información almacenan los bits restantes? Mencione al menos **3 bits de control** y su función.

5.2 Actividad: Simulador de paginación

Guarde como `paging_sim.c`:

```c
// paging_sim.c
#include <stdio.h>

#define PAGE_BITS  4
#define PAGE_SIZE  (1 << PAGE_BITS)              /* 16 bytes/pagina */
#define VA_BITS    8                              /* VA de 8 bits */
#define NUM_PAGES  (1 << (VA_BITS - PAGE_BITS))   /* 16 paginas */

/* Tabla de paginas: -1 = pagina no presente (PAGE FAULT) */
int page_table[NUM_PAGES] = {
    3, -1,  7,  2, -1,  1, -1,  5,
    -1, -1,  4, -1,  6, -1,  0, -1
};

void traducir(int va) {
    int vpn    = va >> PAGE_BITS;
    int offset = va & (PAGE_SIZE - 1);
    printf("VA=0x%02X  VPN=%2d  Offset=%2d  ", va, vpn, offset);
    if (page_table[vpn] == -1) {
        printf("-> PAGE FAULT (pagina no presente)\n");
    } else {
        int pfn = page_table[vpn];
        int pa  = (pfn << PAGE_BITS) | offset;
        printf("-> PFN=%2d  PA=0x%02X\n", pfn, pa);
    }
}

int main() {
    int vas[] = {0x00, 0x0F, 0x20, 0x35, 0x10, 0xA3, 0xC8, 0xF0};
    int n = sizeof(vas) / sizeof(vas[0]);
    printf("%-22s %-6s %-8s %-6s %s\n",
```

```
        "VA", "VPN", "Offset", "PFN", "PA");
        printf("-----------------------------------------------------\n");
        for (int i = 0; i < n; i++) traducir(vas[i]);
        return 0;
    }
```

Code 5: paging_sim.c

5.3 Actividad: Simulador — Análisis

1. Compile y ejecute el simulador. Muestre la salida completa.
2. ¿Qué ocurre con las VAs `0x10` y `0xA3`? ¿Qué debería hacer el SO real ante un *page fault*?
3. ¿Cuántos accesos a memoria física requiere completar una instrucción *load* con tabla de paginas de un solo nivel? ¿Por qué es costoso y que solución de hardware existe?
4. ¿Qué ventaja tiene la paginación sobre la segmentación en cuanto al fenomeno de fragmentación?

6 Gestión de espacio libre

Tema del libro: `vm-freespace` — Free-Space Management

El gestor de memoria libre mantiene una **lista libre** de bloques disponibles en el heap. Las estrategias de asignación principales son: **first fit** (primer bloque que cabe), **best fit** (bloque mas ajustado), **worst fit** (bloque mas grande) y **next fit** (*first fit* desde el ultimo punto de búsqueda).

6.1 Actividad: Simulación de estrategias de asignación

Estado inicial de la lista libre:

Dirección inicio	Tamaño (bytes)
`0x0100`	100
`0x0200`	500
`0x0400`	200
`0x0500`	300
`0x0700`	600

Solicitudes en orden: `malloc(212)` · `malloc(417)` · `malloc(98)` · `malloc(426)`

1. Para cada solicitud indique que bloque asigna **first fit**. Muestre la lista libre resultante tras las 4 asignaciones.
2. Repita con **best fit**. ¿Cambia el resultado?
3. ¿Cuál estrategia genera mas fragmentación externa en este caso? ¿Cuál la minimiza?
4. ¿Qué es el *coalescing*? Ilustre un caso donde su ausencia provoca que una solicitud de 250 bytes falle aunque haya suficiente memoria total libre.
5. ¿Qué es la *fragmentación interna*? ¿Cuándo aparece típicamente al usar un *slab allocator*?

6.2 Actividad: Fragmentación

Guarde como `fragmentation.c`:

```c
// fragmentation.c
#include <stdio.h>
#include <stdlib.h>
#define N 10

int main() {
    void *ptrs[N];
    int sizes[] = {16, 32, 64, 128, 256, 512, 1024, 512, 256, 128};

    /* Asignar N bloques de tamanos variados */
    for (int i = 0; i < N; i++) {
        ptrs[i] = malloc(sizes[i]);
        printf("malloc(%4d)  ->  %p\n", sizes[i], ptrs[i]);
    }

    /* Liberar indices pares para crear huecos */
    printf("\nLiberando bloques en indices pares...\n");
    for (int i = 0; i < N; i += 2) {
        free(ptrs[i]);
        ptrs[i] = NULL;
    }

    /* Intentar asignar un bloque grande */
    void *big = malloc(1500);
    printf("\nmalloc(1500)  ->  %p  [%s]\n",
    big, big ? "exito" : "FALLO");
    if (big) free(big);

    for (int i = 1; i < N; i += 2) free(ptrs[i]);
    return 0;
}
```

Code 6: fragmentation.c

6.3 Actividad: Fragmentación en glibc — Análisis

1. ¿Son consecutivas en memoria las direcciones asignadas? ¿Qué patrón de separación observa entre bloques contiguos?
2. ¿Tiene éxito la asignación final de 1500 bytes? Explique el resultado en términos de fragmentación.
3. Consulta: ¿Cuál es la diferencia entre el *allocator* de usuario (`malloc`/glibc) y el del kernel (*buddy system*, *slab*)? ¿Por qué existen dos niveles de gestión de memoria?

7 TLBs — Translation Lookaside Buffer

Tema: `vm-tlbs` — Faster Translations (TLBs)

El TLB es una **cache de hardware** de traducciones recientes (VPN → PFN). Si la traducción está en el TLB ocurre un **TLB hit** y no se accede a la tabla de paginas en memoria; si no está, ocurre un **TLB miss** y se consulta la tabla. La **localidad espacial** (accesos a paginas cercanas) y la **localidad temporal** (reusar las mismas paginas) maximizan el *hit rate* y minimizan el coste de traducción.

Benchmark de localidad. Guarde como `tlb_locality.c`:

```c
// tlb_locality.c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define N  (1 << 22)   /* 4M enteros = 16 MB */

double ms(struct timespec a, struct timespec b) {
    return (b.tv_sec  - a.tv_sec ) * 1000.0
    + (b.tv_nsec - a.tv_nsec) / 1e6;
}

int main() {
    int *arr = (int *) malloc(N * sizeof(int));
    for (int i = 0; i < N; i++) arr[i] = i;

    struct timespec t0, t1;
    long sum = 0;

    /* Acceso SECUENCIAL --- alta localidad espacial */
```

```c
        clock_gettime(CLOCK_MONOTONIC, &t0);
        for (int i = 0; i < N; i++) sum += arr[i];
        clock_gettime(CLOCK_MONOTONIC, &t1);
        printf("Secuencial : %8.2f ms  (sum=%ld)\n", ms(t0,t1), sum);

        /* Acceso ALEATORIO --- baja localidad (Fisher-Yates) */
        int *idx = (int *) malloc(N * sizeof(int));
        for (int i = 0; i < N; i++) idx[i] = i;
        srand(42);
        for (int i = N-1; i > 0; i--) {
            int j = rand() % (i + 1);
            int t = idx[i]; idx[i] = idx[j]; idx[j] = t;
        }
        sum = 0;
        clock_gettime(CLOCK_MONOTONIC, &t0);
        for (int i = 0; i < N; i++) sum += arr[idx[i]];
        clock_gettime(CLOCK_MONOTONIC, &t1);
        printf("Aleatorio  : %8.2f ms  (sum=%ld)\n", ms(t0,t1), sum);

        free(arr); free(idx);
        return 0;
    }
```

Code 7: tlb_locality.c

Compile el código con opción de optimización:

```
gcc -O0 -o tlb_locality tlb_locality.c
./tlb_locality
```

7.1 Actividad: Localidad y TLB — Análisis

1. ¿Cuántas veces mas lento es el acceso aleatorio frente al secuencial? Muestre el promedio de 3 ejecuciones de `tlb_locality`.

2. Explique con el modelo del TLB por qué el acceso aleatorio es mas lento. ¿Qué ocurre con el *hit rate* en cada caso?

3. Si el tamaño de pagina fuera 64 KB en lugar de 4 KB, ¿mejoraría o empeoraría la situación con accesos aleatorios? Justifique desde el punto de vista del TLB y del uso de memoria.

4. **(Opcional)** Si dispone de `perf`, muestre los contadores `dTLB-load-misses` para ambos patrones (secuencial y aleatorio). ¿Confirman la hipótesis?

   ```
   perf stat -e dTLB-loads,dTLB-load-misses ./tlb_locality
   ```

7.2 Actividad: Comportamiento de los TLB

Responda conceptualmente estas preguntas:

1. Un TLB con 64 entradas (*fully associative*) y paginas de 4 KB. ¿Cuanta memoria puede cubrir sin generar misses? ¿Es suficiente para un proceso moderno típico?

2. Consulte: ¿Qué es un *TLB shootdown* y en que situación ocurre en sistemas multiprocesador? ¿Por qué es una operación costosa?

3. Explique la diferencia entre TLB gestionado por **hardware** (CISC/x86) y por **software** (RISC/MIPS). ¿Cuál ofrece mayor flexibilidad al diseñador del SO y por qué?

Entrega y evaluación

Incluya un enlace a un repositorio de github con:

1. Informe donde se explica claramente la solución (README.md). El informe debe contar con las siguientes secciones:

   (a) Nombres completos de los integrantes, correos y números de documento.
   (b) Documentación de todas las funciones desarrolladas en el código.
   (c) Problemas presentados durante el desarrollo de la práctica y sus soluciones.
   (d) Pruebas realizadas a los programas que verificaron su funcionalidad.
   (e) Un enlace a un video de 10 minutos donde se sustente el desarrollo.
   (f) Manifiesto de transparencia: En que puntos se apoyaron de la IA generativa.
   (g) Al menos cinco conclusiones.

2. Los códigos fuentes de las aplicaciones en una carpeta aparte en el repositorio.

Trabajo a realizar en parejas, o en caso excepcional grupo de 3 personas, evitar trabajar solo. **Entregar un solo repositorio por grupo**.

§·§